

## Experiment No. 8

**Student Name:** Roshan Kumar Singh

**Branch:** MCA(GEN)

**Semester:** 2<sup>nd</sup>

**Subject Name:** Technical Training-1

**UID:** 25MCA20067

**Section/Group:** 25MCA-1\_A

**Date of Performance:** 31/03/26

**Subject Code:** 25CAP-652

### Aim:

Apply stored procedure concepts for database operations

### Objective:

- To understand the concept of stored procedures in PostgreSQL.
- To learn how to pass parameters using IN, OUT, and INOUT modes.
- To perform database operations like updating records using procedures.
- To implement error handling using exception blocks in PL/pgSQL.

### Tools Used:

PostgreSQL

### Procedure:

- Step1: Create the employees4 table and insert sample data.
- Step2: Define the stored procedure update\_salary using PL/pgSQL.
- Step3: Retrieve the employee's current salary and validate existence.
- Step4: Update the salary by adding the given increment.
- Step5: Handle errors using the EXCEPTION block.
- Step6: Execute the procedure using CALL and verify results with SELECT.

**Code:**

```
CREATE TABLE employees4 (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    manager_id INT,  
    department VARCHAR(50),  
    salary INT  
);
```

```
INSERT INTO employees4 VALUES  
(1, 'Amit', NULL, 'Management', 120000),  
(2, 'Ravi', 1, 'Engineering', 80000),  
(3, 'Neha', 1, 'Engineering', 82000),  
(4, 'Karan', 2, 'Engineering', 60000),  
(5, 'Simran', 2, 'Engineering', 62000),  
(6, 'Pooja', 3, 'Engineering', 61000),  
(7, 'Rahul', 3, 'Engineering', 64000),  
(8, 'Arjun', 1, 'HR', 70000);
```

```
CREATE OR REPLACE PROCEDURE update_salary(  
    IN p_emp_id INT,  
    INOUT p_salary NUMERIC(20,3),  
    OUT status VARCHAR(30)  
)  
AS  
$$  
DECLARE  
    curr_sal NUMERIC(20,3);  
BEGIN
```

```
    SELECT salary INTO curr_sal  
    FROM employees4  
    WHERE emp_id = p_emp_id;
```

```
    IF NOT FOUND THEN  
        RAISE EXCEPTION 'EMPLOYEE NOT FOUND';  
    END IF;
```

```
    curr_sal := curr_sal + p_salary;
```

```
UPDATE employees4  
SET salary = curr_sal  
WHERE emp_id = p_emp_id;
```

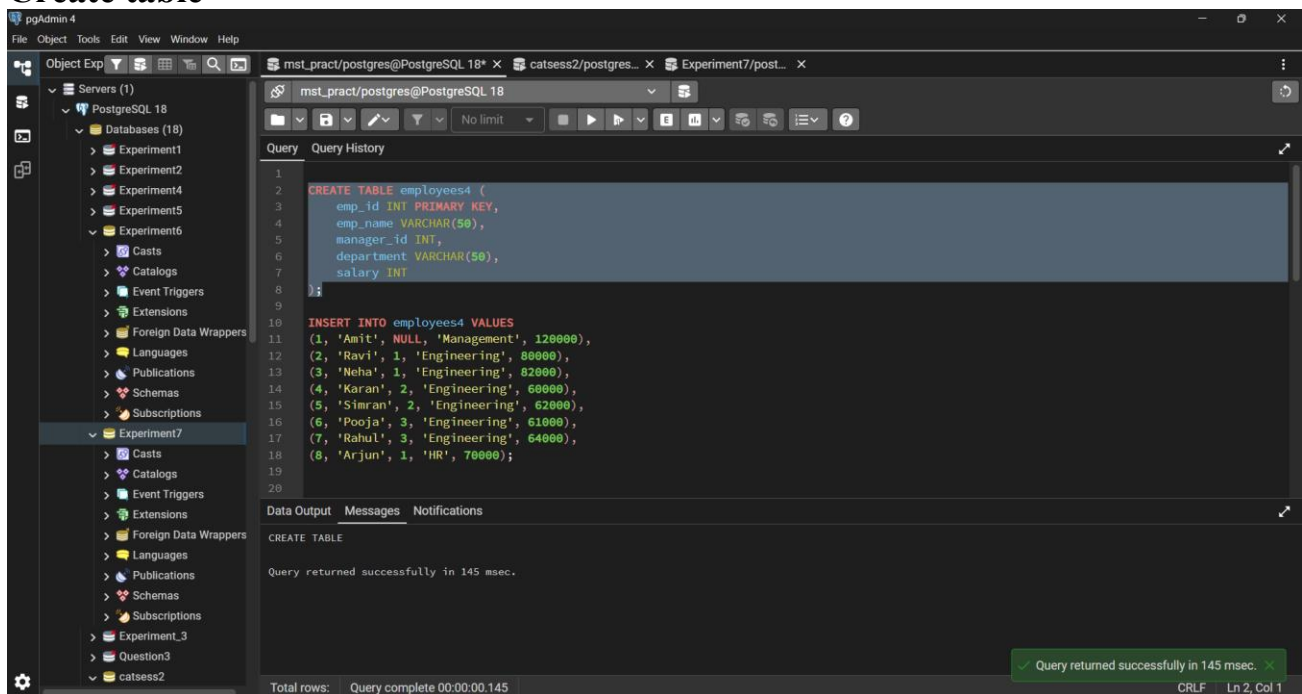
```
p_salary := curr_sal;  
status := 'SUCCESS';
```

```
EXCEPTION  
WHEN OTHERS THEN  
    status := 'FAILED';  
END;  
$$ LANGUAGE plpgsql;
```

```
SELECT * FROM employees4
```

**Output:**

**Create table**

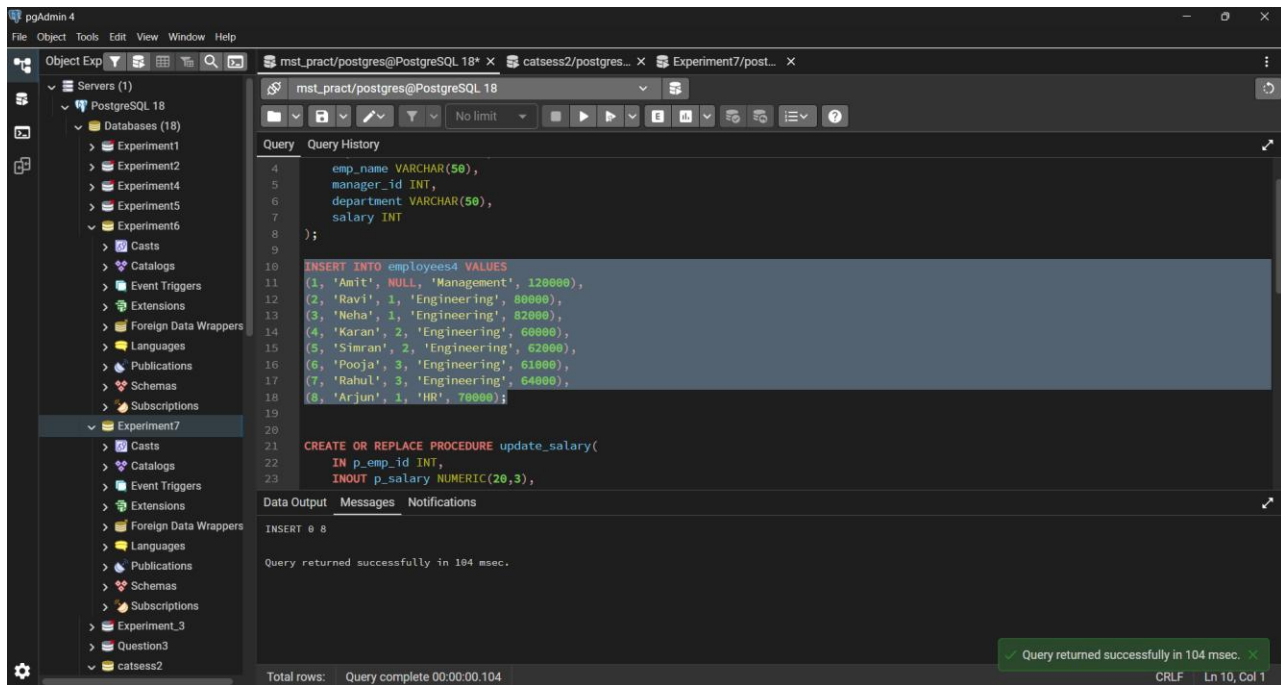


The screenshot displays the pgAdmin 4 web interface. The left sidebar shows a tree view of the database structure, including servers, databases, and various database objects like casts, catalogs, and schemas. The main pane is titled 'mst\_pract/postgres@PostgreSQL 18' and shows a SQL query editor with the following code:

```
1 CREATE TABLE employees4 (  
2   emp_id INT PRIMARY KEY,  
3   emp_name VARCHAR(50),  
4   manager_id INT,  
5   department VARCHAR(50),  
6   salary INT  
7 );  
8  
9  
10 INSERT INTO employees4 VALUES  
11 (1, 'Amit', NULL, 'Management', 120000),  
12 (2, 'Ravi', 1, 'Engineering', 80000),  
13 (3, 'Neha', 1, 'Engineering', 82000),  
14 (4, 'Karan', 2, 'Engineering', 60000),  
15 (5, 'Simran', 2, 'Engineering', 62000),  
16 (6, 'Pooja', 3, 'Engineering', 61000),  
17 (7, 'Rahul', 3, 'Engineering', 64000),  
18 (8, 'Arjun', 1, 'HR', 70000);  
19  
20
```

Below the query editor, the 'Data Output' tab is active, showing the message: 'CREATE TABLE' and 'Query returned successfully in 145 msec.' A green status bar at the bottom right confirms: 'Query returned successfully in 145 msec.' The bottom status bar indicates 'Total rows: Query complete 00:00:00.145' and 'CRLF Ln 2, Col 1'.

## Insert data



The screenshot shows the pgAdmin 4 interface with the 'Query' tab selected. The query editor contains the following SQL code:

```
4 emp_name VARCHAR(50),
5 manager_id INT,
6 department VARCHAR(50),
7 salary INT
8 );
9
10 INSERT INTO employees4 VALUES
11 (1, 'Amit', NULL, 'Management', 120000),
12 (2, 'Ravi', 1, 'Engineering', 80000),
13 (3, 'Neha', 1, 'Engineering', 82000),
14 (4, 'Karan', 2, 'Engineering', 60000),
15 (5, 'Simran', 2, 'Engineering', 62000),
16 (6, 'Pooja', 3, 'Engineering', 61000),
17 (7, 'Rahul', 3, 'Engineering', 64000),
18 (8, 'Arjun', 1, 'HR', 70000);
19
20
21 CREATE OR REPLACE PROCEDURE update_salary(
22 IN p_emp_id INT,
23 INOUT p_salary NUMERIC(20,3),
24 
```

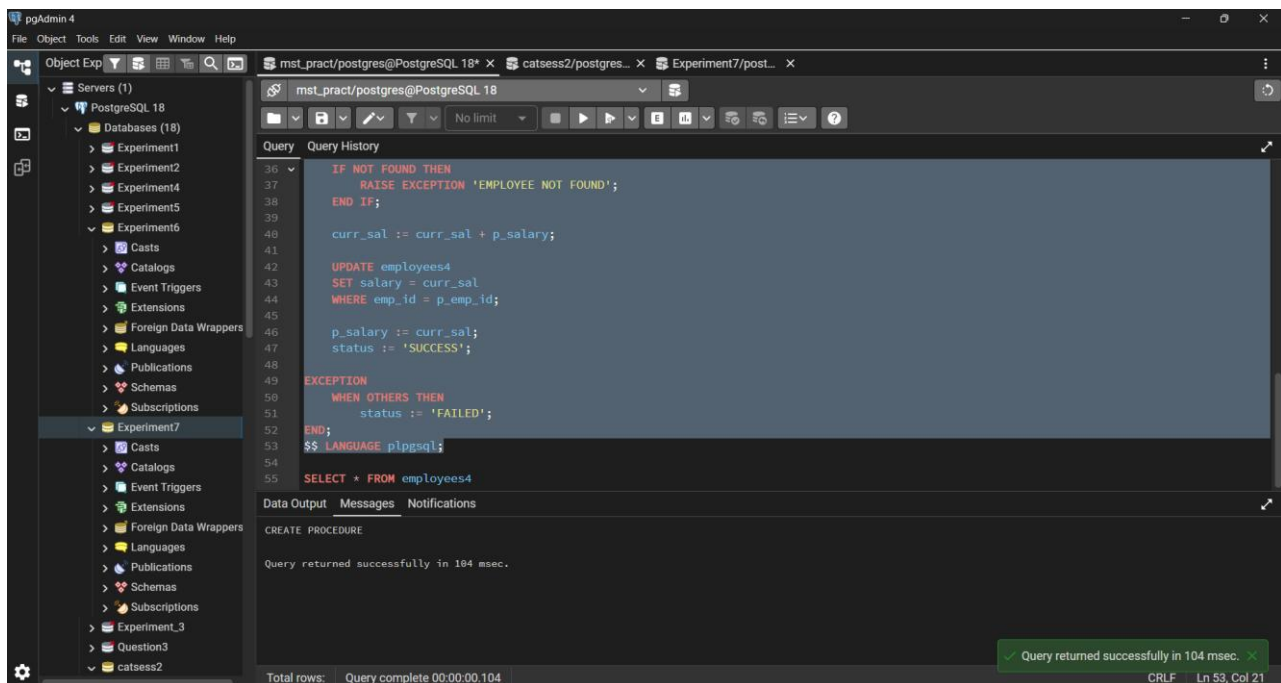
The 'Data Output' tab shows the result of the INSERT statement:

```
INSERT 0 8
```

Query returned successfully in 104 msec.

Total rows: Query complete 00:00:00.104

## Create Procedure



The screenshot shows the pgAdmin 4 interface with the 'Query' tab selected. The query editor contains the following SQL code:

```
36 IF NOT FOUND THEN
37 RAISE EXCEPTION 'EMPLOYEE NOT FOUND';
38 END IF;
39
40 curr_sal := curr_sal + p_salary;
41
42 UPDATE employees4
43 SET salary = curr_sal
44 WHERE emp_id = p_emp_id;
45
46 p_salary := curr_sal;
47 status := 'SUCCESS';
48
49 EXCEPTION
50 WHEN OTHERS THEN
51 status := 'FAILED';
52
53 END;
54 $$ LANGUAGE plpgsql;
55
56 SELECT * FROM employees4
```

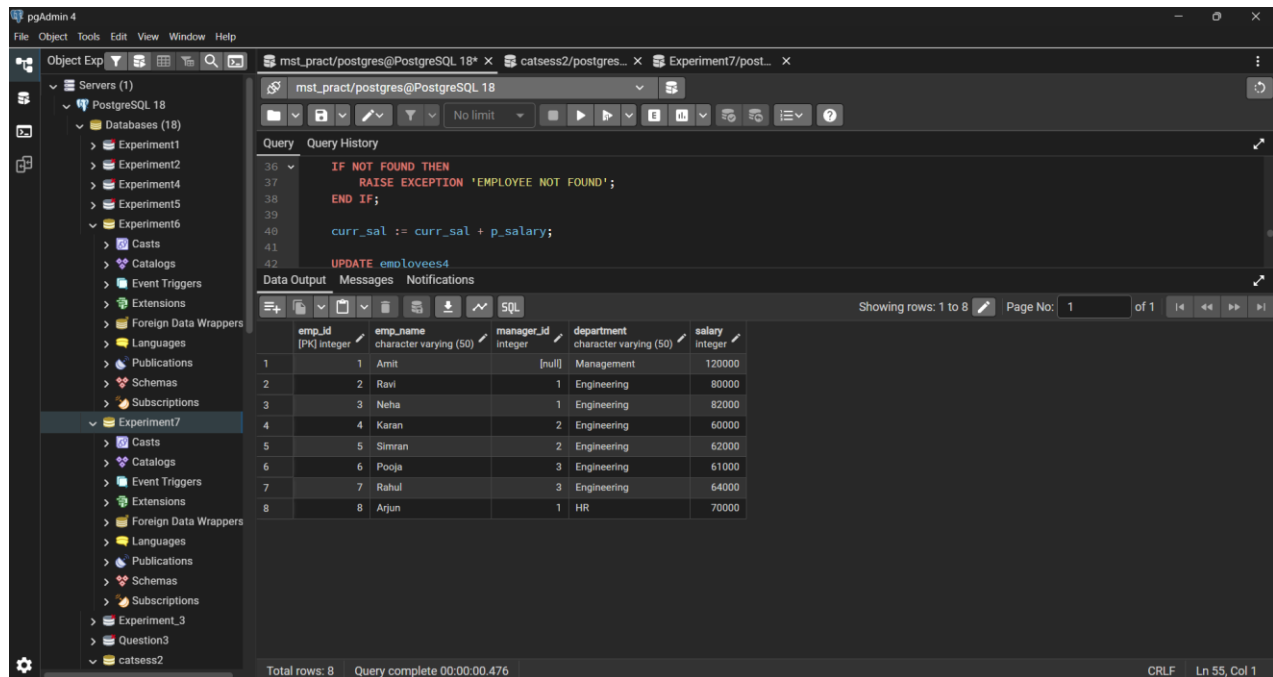
The 'Data Output' tab shows the result of the CREATE PROCEDURE statement:

```
CREATE PROCEDURE
```

Query returned successfully in 104 msec.

Total rows: Query complete 00:00:00.104

## Select Query



The screenshot shows the pgAdmin 4 interface with a PostgreSQL database named 'mst\_pract/postgres@PostgreSQL 18'. The query editor displays the following SQL code:

```

36 IF NOT FOUND THEN
37     RAISE EXCEPTION 'EMPLOYEE NOT FOUND';
38 END IF;
39
40 curr_sal := curr_sal + p_salary;
41
42 UPDATE employees4

```

The 'Data Output' tab shows the results of the query, displaying 8 rows of employee data:

| emp_id | emp_name | manager_id | department  | salary |
|--------|----------|------------|-------------|--------|
| 1      | Amit     | [null]     | Management  | 120000 |
| 2      | Ravi     | 1          | Engineering | 80000  |
| 3      | Neha     | 1          | Engineering | 82000  |
| 4      | Karan    | 2          | Engineering | 60000  |
| 5      | Simran   | 2          | Engineering | 62000  |
| 6      | Pooja    | 3          | Engineering | 61000  |
| 7      | Rahul    | 3          | Engineering | 64000  |
| 8      | Arjun    | 1          | HR          | 70000  |

Total rows: 8 Query complete 00:00:00.476

## Learning Outcomes:

- Understood how to create and use stored procedures in PostgreSQL.
- Learnt the use of IN, OUT, and INOUT parameters in procedures.
- Gained knowledge of updating database records using procedures.
- Understood how to handle exceptions using EXCEPTION block.
- Learnt how to retrieve and modify data dynamically inside procedures.
- Practiced real-world database operations like salary updates.